COM161 - COM161 (75777) - Problem Solving for Computing. 2025-26

Dr. Marwan M Radwan

MRADWAN@QA.COM

Lecture, Week-3

About Me!

- Education
  - BSc in Electrical Engineering with Distinction from BZU, Palestine, 1991
  - Master degree in Computer Engineering, Virginia Tech, VA, USA, 1996
  - PhD degree in Computer Science from University of Leicester, UK, 2017
- Work Experience (more than 30 years of Academic and Professional Experience)
  - Computer Lecturer, QA Higher Education (ZHC), (Oct 21 – Oct24 then Full-Time Since)
  - Postgraduate Supervisor, University of Leicester (2020-2023)
  - Research Associate, University of Leicester (2014-2017)
  - Founder and General Manager, PNINA (.PS ccTLD) (2003-2023)
  - Entrepreneurship and Business consultant, USAID, AFD and IDB (2010-2013)
  - Minister Senior Advisor, Telecom and IT Policy and Planning (2007-2009)
  - Founder and Chairman, Palestinian ICT Association, (2006-2012)
  - Project Manager, Network Senior Consultant, World Bank, , Government Computer Centre, Palestine (1998-2003)

Draft Teaching Plan (2023/24)

Weekly Sessions

Lectures & Tutorials

- *Blend of live timetabled sessions, supported by recorded / directed revision materials*

These sessions will cover:

- Core Theory & self-directed practice
- Review of supplementary topics
- Review of CW / formative assessment
- Regular Q&A
- Practical lab reviews & walkthroughs

- Facilitate formal Attendance / Engagement monitoring

Practical Labs

2hrs of lab-based support

- Supported by the core teaching team and experienced demonstrator(s)
- Practical labs also used to facilitate both formative & summative Class Test assessment

Periodic Targeted Support Sessions

30-45minute live small group tutorials

Blackboard Learn VLE

- Weekly Slides / Gap notes
- Recorded lecture segments & walkthroughs
- Practical lab sheets
- Code samples to support practical labs

- Formal Assessment Outlines
- Assessment Submission Dropboxes
- Class test areas

Ulster University

Module Structure

- Blend of theory & practice
- Practical labs, lectures and assignments well aligned
- Focuses on Procedural Programming via Python
  - Python 3.x, adopting the Standard Library Features

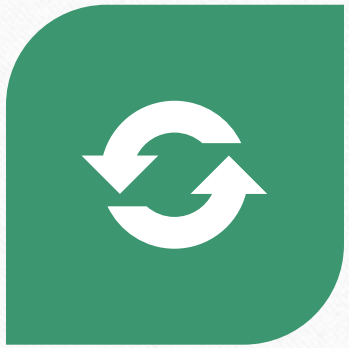
- Core Topics covered:
  - Fundamentals
  - Data
  - Control Structures
  - Logic
  - Structured Programming
  - Data Structures
  - OOP
  - File IO
  - Debugging and Testing
  - Strings

Ulster University

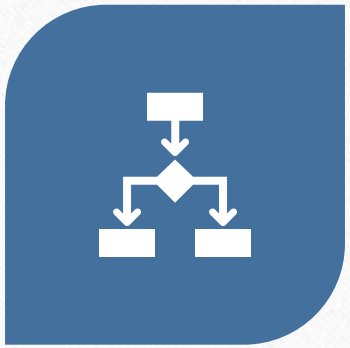
Decision and Logic in Problem Solving

- Programs make decisions
- Logic controls flow
- Core to algorithm design

Control Flow Overview



SEQUENTIAL



SELECTION
(DECISION)

ITERATION
(LOOPS)

Boolean Values

- True
- False
- Produced by comparisons

Comparison Operators

== Equal	!= Not equal	> Greater	< Less
>= Greater or equal	<= Less or equal		

Comparison Example

- x = 10
- y = 5
- print(x > y) # True

Logical Operators

and

or

not

Logical AND Example

- age = 20
- print(age > 18 and age < 30)

Logical OR Example

- score = 45
- print(score < 40 or score > 90)

Logical NOT Example

- is_logged_in = False
- print(not is_logged_in)

10. Truth Tables Concept

AND: both True

OR: at least one True

NOT: reverses value

if Statement Syntax

```
if condition:
    statement
```

Basic if Example

- temp = 30
- if temp > 25:
- print('Hot')

if-else Syntax

- if condition:
- statement
- else:
- statement

if-else Example

- num = 7
- if num % 2 == 0:
- print('Even')
- else:
- print('Odd')

if-elif-else Syntax

- if condition1:
- block1
- elif condition2:
- block2
- else:
- block3

if-elif Example

```
score = 75
if score >= 90:
    print('A')
elif score >= 70:
    print('B')
else:
    print('C')
```

Nested if Statements

```
if x > 0:
    if x % 2 == 0:
        print('Positive Even')
```

Nested if Example

```
x = 8
if x > 0:
    if x % 2 == 0:
        print('Positive Even')
```

Multiple Conditions

- if 10 < x < 20:
- print('Between')

Indentation Rules

Defines blocks

Required in Python

4 spaces standard

Ternary Conditional Expression

- value = 'Even' if num % 2 == 0 else 'Odd'

User
Input
Decision

```
age = int(input())
```

```
if age >= 18:
```

```
    print('Adult')
```

Guarding Against Errors

```
if denominator != 0:
    result = 10 / denominator
```

Combining Conditions

```
if age > 18 and has_ticket:
    print('Entry allowed')
```

Short-Circuit Logic

```
if x != 0 and 10/x > 1:
    print('Valid')
```

Membership Operators

in

not in

```
if 'a' in 'apple':
```

```
print('Found')
```

Identity Operators

- is
- is not

- if x is None:
- print('No value')

Boolean Expressions in Loops

- while x > 0:
- x -= 1

Decision Tree Thinking

Break problem into branches

Evaluate conditions sequentially

Example: Largest of Three Numbers

```
a,b,c = 5,8,3
if a >= b and a >= c:
    print(a)
elif b >= c:
    print(b)
else:
    print(c)
```

Example: Leap Year

- year = 2024
- if (year % 4 == 0 and year % 100 != 0) or year % 400 == 0:
- print('Leap Year')

Example: Password Check

```
password = input()
if password == 'admin123':
    print('Access Granted')
```

Input Validation Pattern

- if not user_input:
- print('Empty input')

Using all() Function

- nums = [2,4,6]
- if all(n % 2 == 0 for n in nums):
- print('All even')

Using any() Function

- nums = [1,3,4]
- if any(n % 2 == 0 for n in nums):
- print('At least one even')

Exception vs Decision

- try:
- x = int(input())
- except ValueError:
- print('Invalid number')

Common Logical Errors

- Wrong operator
- Missing parentheses
- Off-by-one errors

De Morgan's Laws

- not (A and B) == (not A) or (not B)
- not (A or B) == (not A) and (not B)

Problem Solving Strategy

DEFINE PROBLEM

IDENTIFY CONDITIONS

TRANSLATE TO CODE

TEST CASES

Summary

DECISIONS DRIVE LOGIC

BOOLEAN EXPRESSIONS CONTROL FLOW

CLEAR CONDITIONS IMPROVE RELIABILITY